# Java Regex

Mastering Text Pattern Recognition

CODEHIVE

## 1. What is Regex?

A **Regular Expression** (Regex) is a powerful sequence of characters that forms a search pattern. In Java, it is primarily used for text search, data validation (e.g., email or phone checks), and complex string manipulation.

**The Regex Triad:**

- **Pattern Class:** Defines the specific search pattern.
- **Matcher Class:** Performs the search operation.
- **Exception Class:** Handles syntax errors in regex strings.

## 2. Implementation Example

To use Regex in Java, you first compile a pattern and then create a matcher for a specific target string.

```java
import java.util.regex.Matcher;
import java.util.regex.Pattern;

Pattern p = Pattern.compile("codehive", Pattern.CASE_INSENSITIVE);
Matcher m = p.matcher("Welcome to CodeHive Academy!");

if(m.find()) {
    System.out.println("Match found!");
}
```

## 3. Pattern Flags

Flags change how the engine interprets the search. They are passed as the second argument to `Pattern.compile()`.

| Flag | Description |
| --- | --- |
| `CASE_INSENSITIVE` | Ignores 'A' vs 'a'. |
| `LITERAL` | Treats special chars as normal text. |
| `UNICODE_CASE` | Supports non-English case folding. |
| `COMMENTS` | Permits whitespace and comments in regex. |

## 4. Brackets (Character Classes)

Brackets are used to find one character from a specific specified range or set.

| Pattern | Matches |
| --- | --- |
| `[abc]` | Any one character: a, b, or c. |
| `[^abc]` | Any character EXCEPT a, b, or c. |
| `[0-9]` | Any single digit from 0 to 9. |
| `[a-zA-Z]` | Any letter (upper or lower). |

# 5. Metacharacters (Shortcuts)

Metacharacters have special meanings inside the regex engine. Note: In Java strings, you must use a double backslash (e.g., \d).

<table>
<tr><th>Symbol</th><th>Description</th></tr>
<tr><td>.</td><td>Matches any single character.</td></tr>
<tr><td>\d</td><td>Matches any digit [0-9].</td></tr>
<tr><td>\s</td><td>Matches any whitespace (space, tab, newline).</td></tr>
<tr><td>\b</td><td>Matches a word boundary.</td></tr>
<tr><td>|</td><td>Logical OR (e.g., cat|dog).</td></tr>
</table>

## 6. Anchors (Positional)

Anchors do not match characters; they match positions within the string.

| Symbol | Meaning |
| --- | --- |
| ^ | The **start** of the string. |
| $ | The **end** of the string. |

```
// Pattern: "^Admin"
// Matches: "Admin logged in"
// Fails:   "The Admin logged in"
```

## 7. Quantifiers (Frequency)

Quantifiers define how many times the preceding character or group should appear.

| Symbol | Frequency |
| --- | --- |
| n+ | 1 or more times. |
| n* | 0 or more times. |
| n? | 0 or 1 time (Optional). |
| n{x} | Exactly x times. |
| n{x,y} | Between x and y times. |

## 8. Visualizing a Match

Regex matches are often visualized as a pointer moving through the text string.

Target: **"Java 17 is great"**

Pattern: `\d+`

Result: Match at index 5, Value: **"17"**

## 9. Summary & Review

- **Regex** is used for advanced search and replace.
- Always compile the **Pattern** before creating a **Matcher**.
- Use **Metacharacters** like `\d` and `\s` for brevity.
- The `find()` method returns true if the pattern appears anywhere in the target.